Transactional Outbox Pattern

save data and publish an event, reliably

Watch the 4-min video

In one line: writing to the database and publishing to a broker are two separate actions that can't share a transaction, so write the event into an outbox table in the same database transaction as the data, and let a relay or CDC tool publish it; you get at-least-once publishing with no lost events.

1 The dual-write problem

Your order service saves a new order in Postgres, then publishes `OrderPlaced` to Kafka. It works perfectly in testing. Then one day the service crashes right after the database commit and before the publish. The order exists, but shipping never hears about it. Reversing the order (publish first) is no better: the event goes out for an order whose insert then fails. Writing to two systems with no shared transaction is the **dual-write problem**.

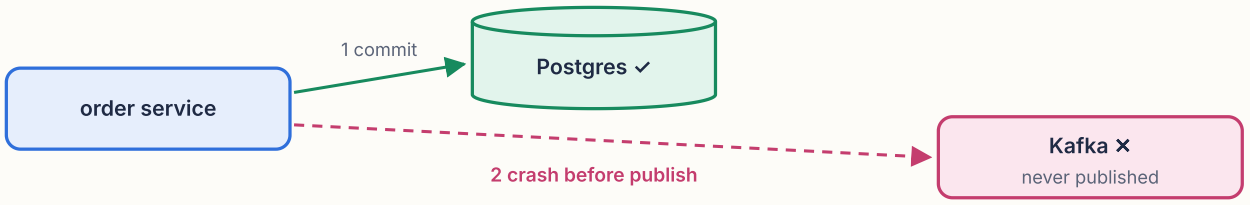


Figure 1. A crash between the two writes leaves the database and the broker disagreeing.

2 The outbox

Add an `outbox` table to the service's own database. In the **same local transaction** that changes business data, insert a row describing the event. Either both commit or neither does. A separate **relay** then reads the outbox and publishes each row to the broker, marking it sent (or relying on CDC offsets).

```
BEGIN;
INSERT INTO orders (id, customer_id, total) VALUES (42, 17, 59.80);
INSERT INTO outbox (event_id, aggregate_id, type, payload)
  VALUES ('e-912', '42', 'OrderPlaced', '{...}');
COMMIT;   -- both rows, or neither
```

Figure 2. The business row and the event row commit together.

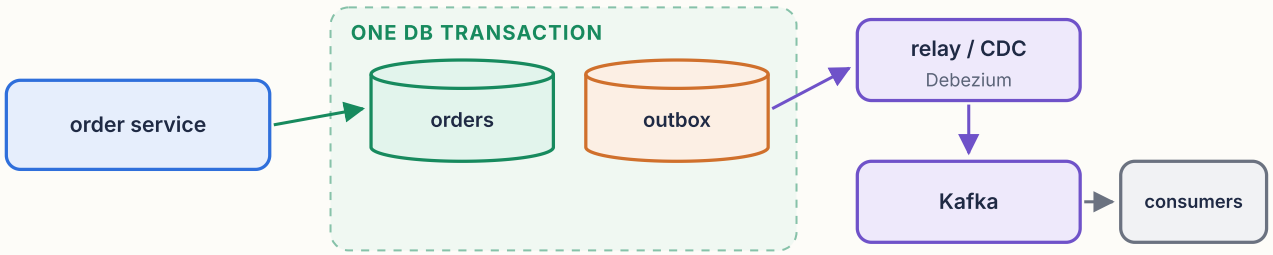


Figure 3. Outbox with CDC: Debezium reads the database log and publishes outbox rows to Kafka.

3 Two ways to relay

- **Polling publisher:** a job selects unsent rows (`FOR UPDATE SKIP LOCKED`), publishes them in order, and marks them sent. Simple; adds some latency and DB load.
- **Log-based CDC:** Debezium tails the database's write-ahead log and turns outbox inserts into Kafka messages (its outbox event router routes by aggregate type and keys by aggregate ID). Low latency, no polling.

Either way, a crash after publishing but before marking means the event is published again: **at-least-once**. Consumers deduplicate by `event_id` (lesson 38). Delete or archive old outbox rows regularly.

WORKED EXAMPLE: ORDERPLACED, RELIABLY

In one transaction, the order service inserts order 42 and an outbox row with event ID `e-912`, type `OrderPlaced`, aggregate ID 42 and the full payload. Debezium sees the outbox insert in Postgres' WAL a few milliseconds later and publishes it to the `orders` topic with key 42. If the service crashes right after the commit, the event still goes out, because it's in the database. If Debezium restarts, it may publish `e-912` again; the shipping service's inbox drops the duplicate.

COMMON MISTAKES

- Publishing inside the transaction (the publish can't roll back).
- Publishing after commit in application code and calling it reliable.
- A relay that publishes out of order for the same aggregate.
- Never cleaning up the outbox table.

4 Key terms

Dual write: updating two systems with no shared transaction

Outbox table: events stored with business data

Relay / message relay: process that publishes outbox rows

CDC: change data capture from the database log

Debezium: open-source CDC platform for Kafka

5 Interview questions

Q1. What problem does the transactional outbox solve? BASIC

The dual-write problem: making sure a database change and its event either both happen or neither, without a distributed transaction.

Q2. How do events get from the outbox to the broker? BASIC

A relay polls the outbox table and publishes unsent rows, or a CDC tool like Debezium reads the database log and publishes the inserts.

Q3. Some orders never reach the shipping service. The code saves, then publishes. Fix it. SCENARIO

Use an outbox: insert the event in the same transaction as the order, relay it with CDC or a poller, and make shipping deduplicate by event ID.

Q4. Polling the outbox every 100 ms loads the DB. Alternatives? SCENARIO

Use log-based CDC (Debezium) instead of polling, or poll less often with batched, indexed queries and SKIP LOCKED.

Q5. Does the outbox give exactly-once publishing? TRICKY

No, at-least-once: a relay can crash after publishing and before recording progress. Consumers must deduplicate.

Q6. How do you keep events for one order in order? TRICKY

Use the aggregate ID as the message key and publish outbox rows in commit order per aggregate.

Next: SD-I-016 · Inbox Pattern and Deduplication backendroadmap.com